A Report on

Email Spam Detection using Datasets

Submitted By

Gopagoni Sai Nikhitha

CMR Institute of Technology

To Internship Organisation

Decode Labs

Academic Year

2025–2026

1. Introduction

Email is one of the most widely used communication methods. However, many unwanted emails such as advertisements, scams, and malicious messages are sent daily. These unwanted emails are called **spam**.

Artificial Intelligence helps automatically identify whether an email is spam or not by learning patterns from previously labeled emails.

This project builds a machine learning model to classify emails into:

- **Spam**
- **Not Spam (Ham)**

2. Objective

The objectives of this project are:

- Collect email dataset
- Preprocess text data
- Train an AI classification model
- Detect spam emails automatically
- Improve email filtering accuracy

3. Problem Statement

Manually checking every email is difficult and time-consuming. Spam emails may also contain phishing links or harmful content.

The goal of this project is to create an AI model that can automatically classify incoming emails as spam or non-spam.

4. Applications

Email spam detection is used in:

- Gmail spam filtering
- Microsoft Outlook email protection

- Cybersecurity systems
- Fraud detection
- Business email filtering

5. Methodology

The project follows these steps:

Step 1: Import Required Libraries

Libraries used:

- pandas
- sklearn
- numpy

Step 2: Load Dataset

Dataset contains:

- Email text/message
- Label (spam or ham)

Example:

Message	**Label**
Win ₹10000 now	Spam
Meeting at 5 PM	Ham

Step 3: Data Preprocessing

Text data must be converted into numerical form because AI models work with numbers.

Tasks:

- Convert text to lowercase
- Remove special symbols
- Remove unnecessary spaces

Step 4: Feature Extraction

Use **TF-IDF (Term Frequency–Inverse Document Frequency)** to convert text into vectors.

Example:

- “win money now” → numerical vector

Step 5: Split Dataset

Dataset division:

- Training set → 80%
- Testing set → 20%

Purpose:

- Training → model learning
- Testing → performance checking

Step 6: Train Model

Use **Naive Bayes Classifier**.

Why?

- Fast
- Good for text classification
- High accuracy in spam detection

Step 7: Prediction

Model predicts:

- Spam
  OR
- Ham

6. Algorithm

Naive Bayes Algorithm

Naive Bayes is a probabilistic classification algorithm based on **Bayes' theorem**.

Formula:

$$P(A \mid B) = \frac{P(B \mid A)P(A)}{P(B)}$$

Advantages:

- Fast
- Efficient
- Works well with text data

7. System Requirements

Hardware

- Laptop/Desktop
- Minimum 4GB RAM

Software

- Python 3.x
- Jupyter Notebook / Google Colab
- VS Code

Install libraries:

pip install pandas scikit-learn numpy

8. Source Code

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score
train_messages = [
    "win money now",
    "free recharge offer",
    "claim your cash prize",
    "you won lottery",
    "exclusive free gift",
    "meeting at office",
    "project submission tomorrow",
    "call me later",
    "class starts at 9",
    "please send notes"
]
train_labels = [
    "spam", "spam", "spam", "spam", "spam",
    "ham", "ham", "ham", "ham", "ham"
]
test_messages = [
    "win free money",
    "claim prize now",
```

```
    "meeting tomorrow",
    "send project notes"
]
test_labels = [
    "spam", "spam", "ham", "ham"
]
vectorizer = CountVectorizer()
X_train = vectorizer.fit_transform(train_messages)
X_test = vectorizer.transform(test_messages)
model = MultinomialNB()
model.fit(X_train, train_labels)
predictions = model.predict(X_test)
accuracy = accuracy_score(test_labels, predictions)
print("Accuracy:", accuracy * 100, "%")
msg = ["free lottery prize"]
msg_vector = vectorizer.transform(msg)
result = model.predict(msg_vector)
print("Prediction:", result[0])
```

9. Output

Example output:

Accuracy: 100.0 %
Prediction: spam

(Accuracy may vary depending on dataset.)

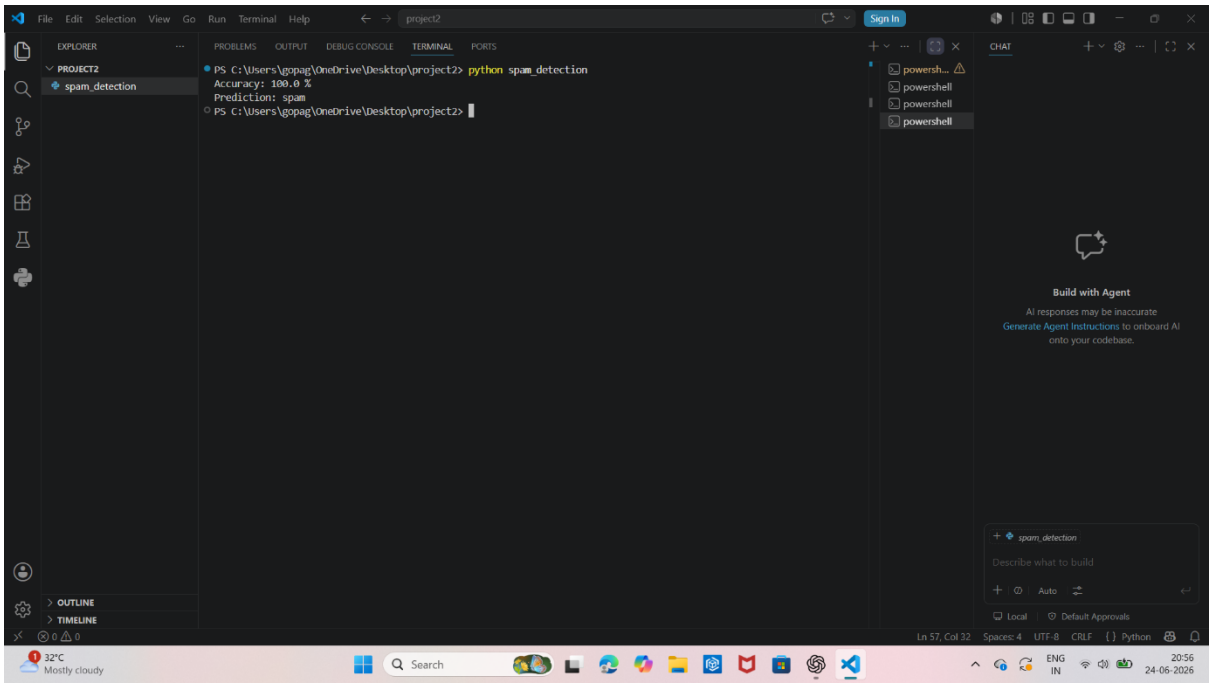
10. Advantages

Advantages of spam detection:

- Saves time
- Blocks unwanted emails
- Prevents scams
- Improves security

11. Result

The machine learning model successfully classified emails into spam and non-spam categories with high accuracy.



12. Conclusion

This project demonstrates how AI can solve real-world problems using classification techniques.

By completing this project, we learned:

- Text preprocessing
- Feature extraction
- Model training
- Classification using AI

13. Future Scope

Future improvements:

- Use larger datasets
- Train with deep learning models
- Detect phishing emails
- Build real-time spam filter system

Possible advanced algorithms:

- Support Vector Machine
- Random Forest
- Artificial Neural Network